

Sơ lược về ứng dụng của hàng đợi trong một lớp bài toán đơn điệu

Tang Ze

Nguồn gốc: A brief analysis of queues in a class of monotonicity problems

Dynamic Programming / Queue Optimization.pdf

1 Ví dụ 1: Chọn vị trí xưởng cưa

Bài CEOI 2004.

Đề bài

Trên một đường thẳng từ đỉnh núi xuống chân núi có n cây. Chính quyền địa phương quyết định chặt chúng. Để không lãng phí gỗ, sau khi cây bị chặt, gỗ phải được vận chuyển tới xưởng cưa. Gỗ chỉ được vận chuyển theo một hướng: xuống núi. Ở chân núi đã có một xưởng cưa; ngoài ra sẽ xây thêm hai xưởng cưa mới trên đường núi. Cần quyết định vị trí xây hai xưởng cưa này sao cho tổng chi phí vận chuyển nhỏ nhất. Giả sử vận chuyển một ki-lô-gam gỗ đi một mét tốn một xu.

Nhiệm vụ là đọc số cây, trọng lượng và vị trí của chúng, tính chi phí vận chuyển nhỏ nhất, rồi in kết quả. Dữ liệu có

$$2 \leq n \leq 20000, \quad 1 \leq v_i \leq 10000, \quad 0 \leq d_i \leq 10000.$$

Cây được đánh số từ đỉnh núi xuống chân núi. Dòng thứ $i + 1$ chứa trọng lượng v_i của cây thứ i và khoảng cách d_i từ cây thứ i tới cây thứ $i + 1$; riêng d_n là khoảng cách từ cây thứ n tới xưởng cưa ở chân núi. Chi phí đưa toàn bộ gỗ về xưởng ở chân núi nhỏ hơn 2 000 000 000 xu.

Ví dụ mẫu có đáp án 26.

Thiết lập

Trước hết cần nhận ra rằng xây xưởng cưa giữa hai cây kề nhau là vô ích. Nếu một xưởng nằm giữa hai cây, ta có thể dời nó lên đúng vị trí cây gần nhất ở phía trên; khi đó chi phí vận chuyển gỗ từ phía trên giảm, còn chi phí của phía dưới không đổi. Vì vậy, chỉ cần xét các vị trí tại cây.

Để tiện thảo luận, giả sử ở chân núi cũng có một cây giả, đánh số $n + 1$, với $v[n + 1] = d[n + 1] = 0$. Định nghĩa

$$\text{sumw}[i] = \sum_{j=1}^i w[j], \quad \text{sumd}[i] = \sum_{j=1}^{i-1} d[j],$$

trong đó $\text{sumd}[1] = 0$.

Gọi $c[i]$ là chi phí nếu xây một xưởng cưa tại cây thứ i và vận chuyển toàn bộ cây từ 1 đến i tới đó. Khi đó

$$c[1] = 0, \quad c[i] = c[i - 1] + \text{sumw}[i - 1]d[i - 1].$$

Gọi $w[j, i]$ là chi phí nếu xây xướng tại cây thứ i và vận chuyển các cây từ j đến i tới đó. Ta có

$$w[j, i] = c[i] - c[j - 1] - \text{sumw}[j - 1](\text{sumd}[i] - \text{sumd}[j - 1]),$$

và $w[j, i] = 0$ khi $i \leq j$.

Tất cả các mảng sumw , sumd và c được tính trong $O(n)$; sau đó mỗi giá trị $w[j, i]$ được tính trong $O(1)$.

Gọi $f[i]$ là chi phí nhỏ nhất khi xướng cửa thứ hai được xây ở cây thứ i . Khi đó

$$f[i] = \min_{1 \leq j \leq i-1} \{c[j] + w[j + 1, i] + w[i + 1, n + 1]\}.$$

Thuật toán trực tiếp mất $O(n^2)$, không thể dùng cho kích thước dữ liệu này.

Tính đơn điệu của quyết định

Ta chứng minh nhận xét sau. Nếu khi xây xướng cửa thứ hai ở vị trí i , vị trí tối ưu của xướng thứ nhất là j , thì khi xây xướng thứ hai ở vị trí $i + 1$, vị trí tối ưu của xướng thứ nhất không nhỏ hơn j .

Giả sử j là vị trí nhỏ nhất để $f[i]$ đạt tối ưu:

$$f[i] = c[j] + w[j + 1, i] + w[i + 1, n + 1].$$

Với mọi $k < j$, ta có

$$c[k] + w[k + 1, i] + w[i + 1, n + 1] > c[j] + w[j + 1, i] + w[i + 1, n + 1],$$

nên

$$c[k] + w[k + 1, i] > c[j] + w[j + 1, i].$$

Khi mở rộng từ i sang $i + 1$, phần gỗ từ $k + 1$ đến i nặng hơn phần gỗ từ $j + 1$ đến i vì $k < j$. Do đó sau khi cộng thêm chi phí vận chuyển qua đoạn $d[i]$, quyết định k vẫn kém j :

$$c[k] + w[k + 1, i + 1] > c[j] + w[j + 1, i + 1].$$

Cộng thêm cùng một lượng $w[i + 2, n + 1]$, ta thấy khi tính $f[i + 1]$, quyết định k vẫn kém quyết định j . Vì mọi $k < j$ đều bị loại, quyết định tối ưu đầu tiên của $f[i + 1]$ phải lớn hơn hoặc bằng j .

Cải tiến bằng hàng đợi

Đặt $s[k, i]$ là giá trị của hàm mục tiêu khi biến quyết định bằng k :

$$s[k, i] = c[k] + w[k + 1, i] + w[i + 1, n + 1].$$

Với $k_1 < k_2$, ta có

$$\begin{aligned} s[k_1, i] - s[k_2, i] &= \text{sumd}[i](\text{sumw}[k_2] - \text{sumw}[k_1]) \\ &\quad - (\text{sumw}[k_2] \text{sumd}[k_2] - \text{sumw}[k_1] \text{sumd}[k_1]). \end{aligned}$$

Nếu $s[k_1, i] - s[k_2, i] < 0$, thì

$$\frac{\text{sumw}[k_2] \text{sumd}[k_2] - \text{sumw}[k_1] \text{sumd}[k_1]}{\text{sumw}[k_2] - \text{sumw}[k_1]} > \text{sumd}[i].$$

Ký hiệu vế trái là $g[k_1, k_2]$. Khi $g[k_1, k_2] > \text{sumd}[i]$, quyết định k_1 tốt hơn k_2 tại trạng thái i .

Do quyết định tối ưu đơn điệu, ta duy trì một hàng đợi các quyết định $k_1 < k_2 < \dots < k_p$ sao cho

$$g[k_1, k_2] < g[k_2, k_3] < \dots < g[k_{p-1}, k_p].$$

Trước khi tính $f[i]$, nếu $g[k_1, k_2] < \text{sumd}[i]$, nghĩa là k_1 không tốt bằng k_2 , ta xóa k_1 khỏi đầu hàng đợi, và lặp lại cho đến khi đầu hàng đợi là quyết định tốt nhất. Khi đó

$$f[i] = c[k_1] + w[k_1 + 1, i] + w[i + 1, n + 1]$$

được tính trong $O(1)$.

Sau khi tính $f[i]$, ta chèn quyết định mới i vào cuối hàng đợi. Nếu

$$g[k_{p-1}, k_p] > g[k_p, i],$$

thì k_p trở nên vô ích: trước khi k_p tốt hơn k_{p-1} , quyết định i đã tốt hơn k_p . Vì vậy ta xóa k_p và lặp lại cho đến khi thứ tự các giá trị g được khôi phục. Mỗi quyết định vào hàng đợi một lần và ra hàng đợi một lần, nên tổng chi phí duy trì hàng đợi là $O(n)$. Do đó thuật toán chạy trong $O(n)$.

Điều kiện quan trọng trong ví dụ này là trọng lượng mỗi cây đều dương. Nhờ đó mảng tiền tố trọng lượng đơn điệu, và lập luận “đoạn từ $j + 1$ đến i nhẹ hơn đoạn từ $k + 1$ đến i khi $k < j$ ” mới đúng.

2 Ví dụ 2: Xây kho

Bài tuyển chọn tỉnh Chiết Giang năm 2007.

Đề bài

Công ty L có N nhà máy nằm trên một ngọn núi, đánh số từ cao xuống thấp: nhà máy 1 ở đỉnh núi, nhà máy N ở chân núi. Công ty thường để sản phẩm ngoài trời để tiết kiệm chi phí, nhưng do sắp có mưa lớn, họ cần xây một số kho tại các nhà máy. Nhà máy thứ i hiện có P_i sản phẩm; chi phí xây kho tại đó là C_i . Sản phẩm của nhà máy không có kho phải được vận chuyển xuống các kho ở phía dưới, tức chỉ có thể đi tới nhà máy có chỉ số lớn hơn. Chi phí vận chuyển một sản phẩm qua một đơn vị khoảng cách là 1, và mỗi kho có sức chứa đủ lớn.

Dữ liệu gồm khoảng cách X_i từ nhà máy 1 đến nhà máy i ($X_1 = 0$), số sản phẩm P_i và chi phí xây kho C_i . Cần tìm phương án xây kho sao cho tổng chi phí xây dựng và vận chuyển nhỏ nhất. Kích thước dữ liệu có thể tới $N \leq 10^6$, các số nằm trong kiểu số nguyên có dấu 32 bit, và kết quả trung gian không vượt quá kiểu số nguyên có dấu 64 bit.

Phân tích ban đầu

Đặt

$$\text{sump}[i] = \sum_{j=1}^i P_j,$$

và $\text{sumw}[i]$ là chi phí vận chuyển toàn bộ sản phẩm từ nhà máy 1 đến i về nhà máy i . Gọi $w[i, j]$ là chi phí vận chuyển sản phẩm từ nhà máy i đến j về nhà máy j . Khi đó

$$w[i, j] = \text{sumw}[j] - \text{sumw}[i - 1] - \text{sump}[i - 1](x[j] - x[i - 1]).$$

Các giá trị tiền tố được tính trong $O(n)$, và mỗi $w[i, j]$ được tính trong $O(1)$.

Gọi $f[i]$ là chi phí nhỏ nhất khi xây một kho tại nhà máy i và xử lý xong các nhà máy từ 1 đến i . Đáp án là $f[n]$, với

$$f[i] = \min_{0 \leq k \leq i-1} \{f[k] + w[k+1, i] + c[i]\}, \quad f[0] = 0.$$

Thuật toán trực tiếp có độ phức tạp $O(n^2)$. Dưới đây dùng cùng tư tưởng như ví dụ thứ nhất để giảm xuống $O(n)$.

Đơn điệu và tối ưu hóa

Giả sử quyết định tối ưu khi tính $f[i]$ là k . Với mọi $j < k$, từ tính tối ưu của k ta có

$$f[k] + w[k+1, i] + c[i] < f[j] + w[j+1, i] + c[i].$$

Viết lại bằng các mảng tiền tố:

$$f[k] - \text{sumw}[k] + \text{sump}[k](x[i] - x[k]) < f[j] - \text{sumw}[j] + \text{sump}[j](x[i] - x[j]).$$

Vì $\text{sump}[k] \geq \text{sump}[j]$ và $x[i+1] \geq x[i]$, bất đẳng thức vẫn đúng khi thay $x[i]$ bằng $x[i+1]$. Do đó quyết định k cũng tốt hơn j khi xét trạng thái $i+1$. Suy ra quyết định tối ưu đơn điệu theo i .

Đặt

$$s[k, i] = f[k] + w[k+1, i] + c[i].$$

Với $k_1 < k_2$,

$$\begin{aligned} s[k_1, i] - s[k_2, i] &= (f[k_1] + \text{sumw}[k_1] + \text{sump}[k_1]x[k_1]) \\ &\quad - (f[k_2] + \text{sumw}[k_2] + \text{sump}[k_2]x[k_2]) \\ &\quad - x[i](\text{sump}[k_1] - \text{sump}[k_2]). \end{aligned}$$

Khi $s[k_1, i] - s[k_2, i] < 0$, ta nhận được một ngưỡng chỉ phụ thuộc vào k_1, k_2 :

$$g[k_1, k_2] = \frac{f[k_1] + \text{sumw}[k_1] + \text{sump}[k_1]x[k_1] - f[k_2] - \text{sumw}[k_2] - \text{sump}[k_2]x[k_2]}{\text{sump}[k_1] - \text{sump}[k_2]} > x[i].$$

Vì vậy có thể dùng hàng đợi đơn điệu tương tự ví dụ trước: xóa đầu hàng đợi khi quyết định kế tiếp đã tốt hơn tại $x[i]$, dùng đầu hàng đợi để tính $f[i]$, rồi chèn quyết định i ở cuối và xóa các quyết định bị nó làm cho vô dụng. Tổng thời gian vẫn là $O(n)$.

3 Ví dụ 3: Vũ điệu lộng lẫy

Bài NOI 2005 ngày 1.

Đề bài

Cho một ma trận N hàng, M cột; một số ô là chướng ngại vật. Một người bắt đầu trượt từ một ô cho trước. Mỗi lần trượt có một hướng và được đi liên tục tối đa c ô theo hướng đó, cũng có thể đứng yên. Các lần trượt có thể có giá trị c khác nhau. Trong khi trượt không được chạm chướng ngại vật. Cho theo thứ tự K lần trượt, mỗi lần gồm hướng đông, nam, tây hoặc bắc và giá trị c , hãy tìm quãng đường dài nhất có thể trượt được, tính bằng số ô.

Dữ liệu có

$$1 \leq N, M \leq 200, \quad K \leq 200, \quad \sum_{i=1}^K c_i \leq 40000.$$

Quy hoạch động trực tiếp

Gọi $f(k, x, y)$ là quãng đường dài nhất sau k lần trượt và đang ở ô (x, y) . Chỉ xét hướng đông; ba hướng còn lại suy ra tương tự. Ta có

$$f(0, \text{startx}, \text{starty}) = 0,$$

và

$$f(k, x, y) = \max\{f(k-1, x, y), \\ f(k-1, x, y-1) + 1, \\ f(k-1, x, y-2) + 2, \dots, \\ f(k-1, x, y') + y - y'\},$$

trong đó y' là vị trí xa nhất bên trái còn có thể trượt tới mà không vượt quá c_k ô và không đi qua chướng ngại vật.

Số trạng thái là $O(KMN)$, nhưng mỗi lần chuyển có thể mất $O(\max\{M, N\})$, nên tổng thời gian là $O(KMN \max\{M, N\})$, quá lớn.

Biến đổi công thức

Không xét chướng ngại vật trước. Khi tính $f(k, x, y)$ và $f(k, x, y+1)$, rất nhiều trạng thái cũ bị xét lặp lại. Với một hàng cố định và hướng đông, công thức có dạng

$$f(k, x, y) = \max\{f(k-1, x, t) + y - t \mid y - c_k \leq t \leq y\}.$$

Đặt

$$a_t = f(k-1, x, t) - t + 1.$$

Khi đó các ứng viên chỉ khác nhau bởi cùng một lượng phụ thuộc vào y , nên bài toán trở thành lấy giá trị lớn nhất của một đoạn liên tiếp trên dãy a_t .

Khi thêm chướng ngại vật, mỗi hàng được chia thành các đoạn không có vật cản. Trong quá trình quét từ trái sang phải, tập chỉ số hợp lệ luôn là một đoạn: mỗi bước thêm một giá trị ở cuối, đôi khi xóa vài giá trị ở đầu vì độ dài vượt quá $c_k + 1$, và khi gặp vật cản thì xóa sạch đoạn hiện tại. Có thể dùng cây đoạn để hỏi giá trị lớn nhất, đạt thời gian $O(KMN \log \max\{M, N\})$, nhưng cách này vừa dài vừa có hằng số lớn.

Hàng đợi đơn điệu

Tính chất của đoạn đang quét phù hợp với hàng đợi: chỉ thêm ở một đầu và xóa ở đầu kia. Vấn đề là cần lấy giá trị lớn nhất trong hàng đợi. Ta lưu hàng đợi theo dạng đơn điệu giảm của giá trị a .

Nếu trong hàng đợi đã có hai phần tử a_2 và a_3 với $a_2 \leq a_3$, thì a_2 không bao giờ cần dùng. Khi a_3 đã xuất hiện, trước khi a_2 bị xóa thì a_3 cũng vẫn còn trong hàng đợi; vì $a_2 \leq a_3$, giá trị lớn nhất không thể là a_2 . Vì vậy khi chen một phần tử mới a_i , ta xóa từ cuối hàng đợi mọi phần tử không lớn hơn a_i , rồi mới chen i .

Khi xử lý ô hiện tại, nếu chỉ số ở đầu hàng đợi cách ô hiện tại quá c_k , ta xóa nó khỏi đầu. Giá trị lớn nhất trong cửa sổ hiện tại luôn nằm ở đầu hàng đợi, nên truy vấn mất $O(1)$. Mỗi phần tử được chen nhiều nhất một lần và xóa nhiều nhất một lần trên mỗi hàng hoặc cột, do đó tổng thời gian cho một lần trượt là $O(MN)$, và toàn bộ thuật toán chạy trong $O(KMN)$.

Với hướng đông, mã giả có thể viết như sau:

```
khởi tạo  $f[0][x][y] = -\infty$  cho mọi  $x, y$ ,  
 $f[0][startx][starty] = 0$ ;  
for  $k = 1 \dots K$  :  
  for  $x = 1 \dots n$  :  
    xóa rỗng hàng đợi;  
    for  $y = 1 \dots m$  :  
      nếu  $(x, y)$  là chướng ngại, đặt  $f[k][x][y] = -\infty$  và xóa hàng đợi;  
      nếu đầu hàng đợi  $< y - c_k$ , xóa đầu;  
      xóa cuối khi giá trị ở cuối không lớn hơn  $f[k-1][x][y] - y + 1$ ;  
      chèn  $y$ ;  
       $f[k][x][y] = f[k-1][x][front] + y - front$ .
```

Các hướng còn lại chỉ thay đổi thứ tự quét và cách tính chỉ số.

4 Kết luận

Nhìn lại toàn bộ quá trình, ta thường bắt đầu bằng một lời giải quy hoạch động tự nhiên. Khi mỗi chuyển trạng thái quá đắt, bước quan trọng là nhận ra các phép tính lặp và tìm cấu trúc đơn điệu nằm trong biến quyết định hoặc trong cửa sổ chuyển trạng thái. Cây đoạn là một công cụ tổng quát tốt, nhưng trong nhiều trường hợp hàng đợi đơn điệu đơn giản hơn, nhanh hơn và ít lỗi hơn khi cài đặt.

Điều này cho thấy trong xu hướng mới của các kỳ thi tin học, vai trò của cấu trúc dữ liệu cơ bản không hề giảm đi; ngược lại, chúng ngày càng quan trọng. Khi đã tìm được một thuật toán tốt nhưng việc cài đặt quá phức tạp, nên thử đổi góc nhìn và dùng các cấu trúc dữ liệu cơ bản. Nhiều khi cách này đem lại hiệu quả gấp bội.